CIBERTEC

Programa IA Generativa Desarrollo de Soluciones con Python

Unidad 6: Arquitectura de sistemas basados en LLMs

OBJETIVOS

- Diseñar sistemas basados en LLMs.
- Diferenciar entre sistemas LLM online y offline.
- Implementar bases de datos vectoriales en sistemas LLM.

Módulo 18: Sistemas online vs. offline

AGENDA

1. Comparación entre soluciones online y offline
2. Casos de uso y aplicaciones prácticas

Laboratorio:

Host de app en Streamlit con LLM cache **(Evaluación)**

1. COMPARACIÓN ENTRE SOLUCIONES ONLINE Y OFFLINE

1. COMPARACIÓN ENTRE SOLUCIONES ONLINE Y OFFLINE

	Online machine learning	Offline machine learning
Complexity	More complex because the model keeps evolving over time as more data becomes available.	Less complex because the model is fed with more consistent data sets periodically.
Computational power	More computational power is required because of the continuous feed of data that leads to continuous refinement.	Fewer computational power is needed because data is delivered in batches; the model isn't continuously refining itself.
Use in production	Harder to implement and control because the production model changes in real-time according to its data feed.	Easier to implement because offline learning provides engineers with more time to perfect the model before deployment.
Applications	Used in applications where new data patterns are constantly required (e.g., weather prediction tools)	Used in applications where data patterns remain constant and don't have sudden concept drifts (e.g., image classification)

2. CASOS DE USO Y APLICACIONES PRÁCTICAS

Caso de Uso	Aprendizaje Automático en Línea	Aprendizaje Automático Fuera de Línea (Por Lotes)
Detección de Spam	Actualiza continuamente los filtros de spam según los nuevos correos electrónicos recibidos.	Entrena el modelo periódicamente utilizando datos históricos de correos electrónicos para detectar spam.
Sistemas de Recomendación	Actualiza las preferencias del usuario y las recomendaciones en tiempo real a medida que los usuarios interactúan con el contenido.	Analiza las interacciones históricas de los usuarios para crear recomendaciones personalizadas.
Predicción del Mercado de Valores	Adapta los modelos a las condiciones cambiantes del mercado a medida que llegan nuevos datos.	Entrena modelos con datos históricos del mercado de valores para predecir tendencias.
Predicción de Clics en Anuncios	Actualiza los modelos de recomendación de anuncios según las interacciones del usuario.	Entrena modelos utilizando datos históricos de clics en anuncios para predecir futuros clics.
Clasificación de Imágenes	-	Entrena el modelo en un gran conjunto de datos de imágenes etiquetadas para una clasificación precisa.
Segmentación de Clientes	-	Analiza datos históricos de compras para agrupar a los clientes según su comportamiento.
Detección de Fraude	-	Entrena modelos periódicamente utilizando datos históricos de transacciones para detectar anomalías.
Traducción de Idiomas	-	Entrena modelos de traducción en corpus de texto paralelo para obtener traducciones precisas.

LABORATORIO Nº 15:
APP EN STREAMLIT CON LLM CACHE

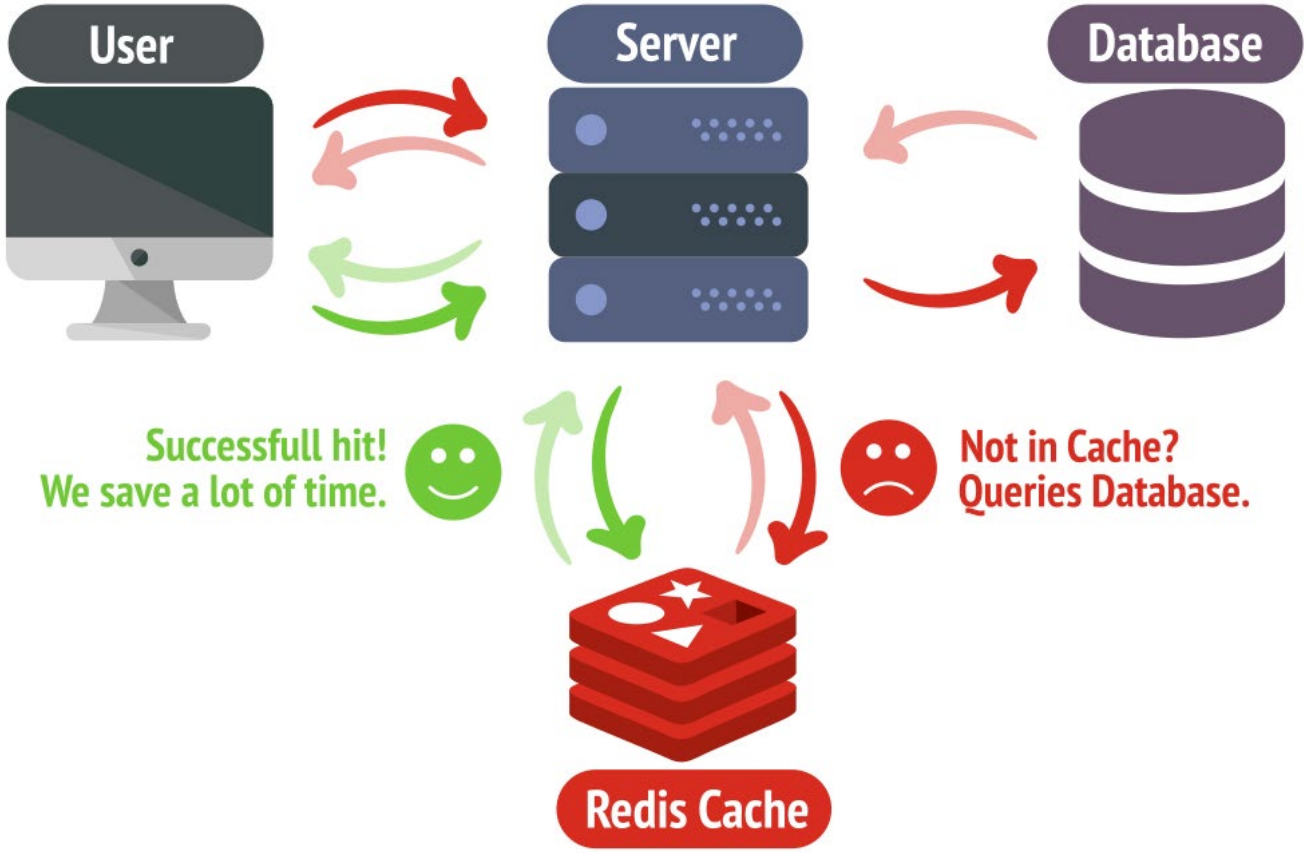
En este laboratorio, podrás aplicar tus conocimientos en FastAPI y Docker para desarrollar una aplicación que utilice el API de GPT y Redis como sistema de caché para LLMs.

Tarea 1: Levantar una aplicación en FastAPI que realice una tarea de análisis sencilla mediante el API de GPT.

Tarea 2: Integrar una instancia de Redis mediante Docker Compose que funcione como un caché de solicitudes para el sistema de LLM.

Tarea Opcional: Registrar el uso y configuración de la caché en la respuesta del API.

LLM Caché


User
Server
Database
Successfull hit!
We save a lot of time.
Not in Cache?
Queries Database.
Redis Cache

Módulo 19: ETL

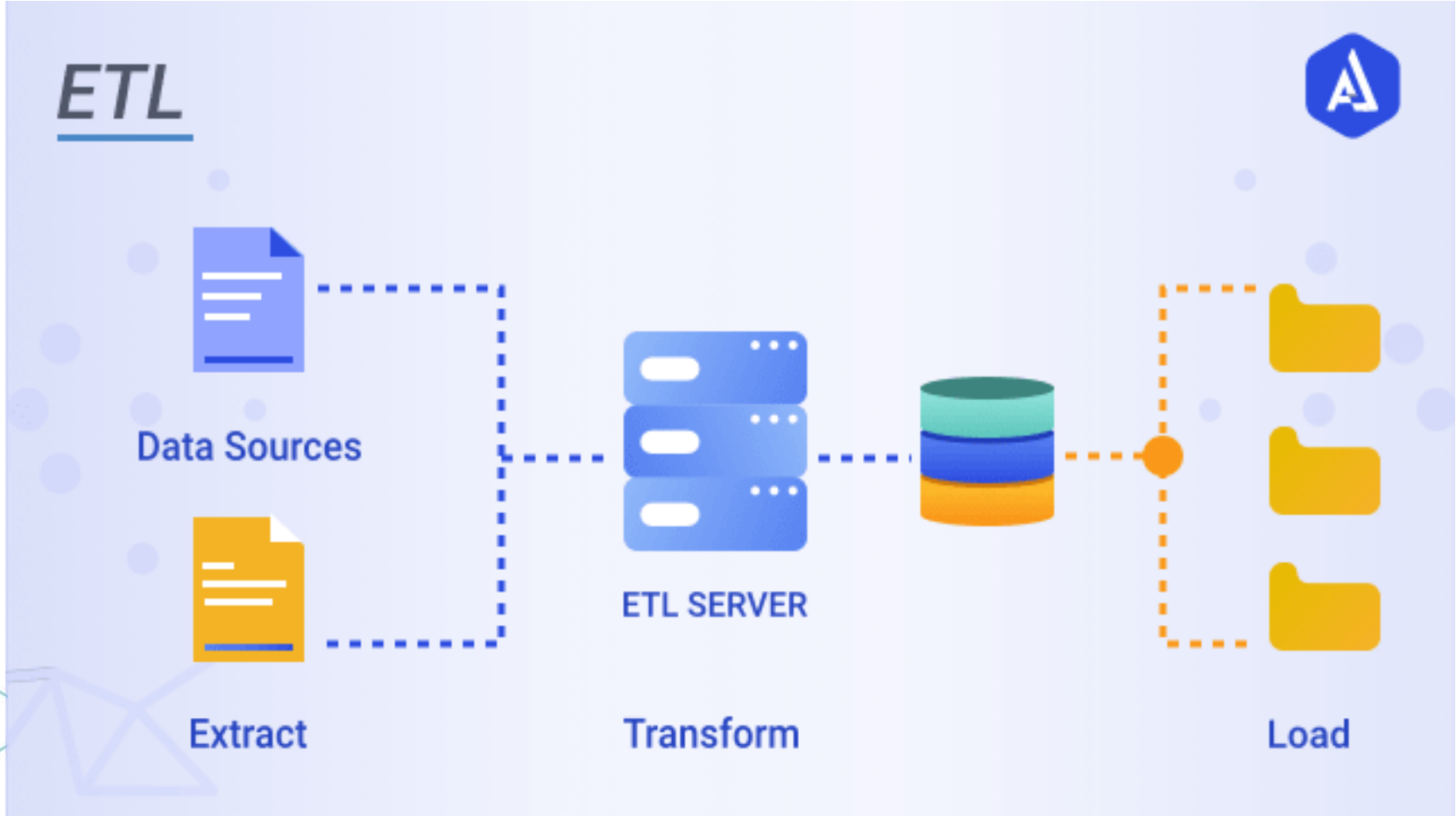
AGENDA

1. Conceptos básicos de ETL y su importancia en la gestión de datos
2. Integración de procesos ETL en sistemas basados en LLMs.
3. Introducción a la Idempotencia en ETL
4. Aplicación del paralelismo en ETL

Laboratorio:

ETL Básico con Airflow **(Evaluación)**

1. CONCEPTOS BÁSICOS DE ETL Y SU IMPORTANCIA EN LA GESTIÓN DE DATOS



1. CONCEPTOS BÁSICOS DE ETL Y SU IMPORTANCIA EN LA GESTIÓN DE DATOS

Las herramientas de ETL ofrecen:

- Automatización integral y facilidad de uso.
- Una interfaz visual de arrastrar y soltar.
- Soporte para gestión de datos complejos.
- Seguridad y cumplimiento.

1. CONCEPTOS BÁSICOS DE ETL Y SU IMPORTANCIA EN LA GESTIÓN DE DATOS

¿Por qué Airflow?

- Ideal para la orquestación de flujos de datos.
- Extensible para conectarse con nuevas tecnologías.
- Si prefieres el código sobre los clics, el enfoque basado en Python de Airflow es ideal.

Apache Airflow

1. CONCEPTOS BÁSICOS DE ETL Y SU IMPORTANCIA EN LA GESTIÓN DE DATOS

En Airflow, los flujos de trabajo se definen mediante código Python. Esto ofrece varias ventajas:

- **Pipelines dinámicos:** los pipelines de Airflow pueden generarse dinámicamente.
- **Extensibilidad:** el framework proporciona operadores para conectarse con diversas tecnologías y todos los componentes son extensibles.
- **Parametrización:** la parametrización del flujo de trabajo está incorporada mediante el motor de plantillas Jinja.

Apache Airflow

2. INTEGRACIÓN DE PROCESOS ETL EN SISTEMAS BASADOS EN LLMS

- Airflow – Operador de conexión a OpenAI

  https://airflow.apache.org/docs/apache-airflow-providers-openai/stable/_api/airflow/providers/openai/operators/openai/index.html

- Conexión desde Python

3. INTRODUCCIÓN A LA IDEMPOTENCIA EN ETL

Idempotente significa que la tarea se puede ejecutar de manera segura múltiples veces sin cambiar el resultado después de la primera ejecución.

- Mantiene los datos consistentes.
- Simplifica el manejo de errores.
- Optimiza el uso de recursos.
- Hace los sistemas más confiables y eficientes.

4. APLICACIÓN DE PARALELISMO EN UN ETL

Concurrencia: ejecución en paralelo de tus tareas

Formas de manejar el paralelismo:

- **A nivel de DAG**
  - Parámetro `concurrency`: Cantidad de tareas que pueden ejecutarse al mismo tiempo para un DAG dado (a través de ejecuciones del DAG)
  - Parámetro `max_active_runs`: Cuantas ejecuciones del DAG pueden darse al mismo tiempo

- **A nivel de tarea**
  - Parámetro `task_concurrency`: Concurrencia máxima para la tarea (a través de todas las ejecuciones del DAG)

LABORATORIO Nº 16: ETL BÁSICO CON AIRFLOW

En este laboratorio explorarán cómo implementar y gestionar los Directed Acyclic Graphs (DAGs) en Airflow. Los estudiantes trabajarán en un proyecto local usando el entorno de desarrollo de Visual Studio Code y la plataforma Astronomer para orquestar y ejecutar sus flujos de trabajo Airflow.

- **Tarea 1:** Configuración del entorno de desarrollo local con Airflow y Astronomer.
- **Tarea 2:** Creación de DAGs en Airflow.
- **Tarea 1:** Gestión y optimización de DAGs en Airflow.

Módulo 20: Framework para desarrollo de aplicaciones basadas en LLMs

AGENDA

1. Introducción a los orquestadores de LLMs
2. Características y funcionalidades de LangChain
3. Cadenas de LangChain

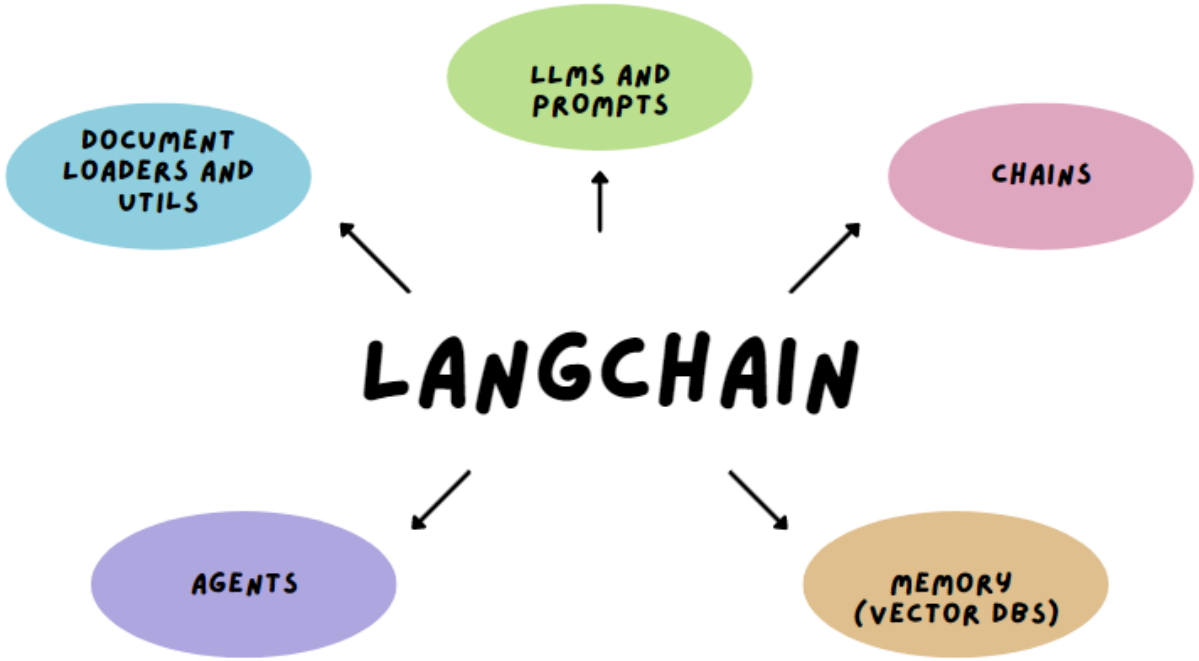
Laboratorio:

Orquestación con Langchain **(Evaluación)**

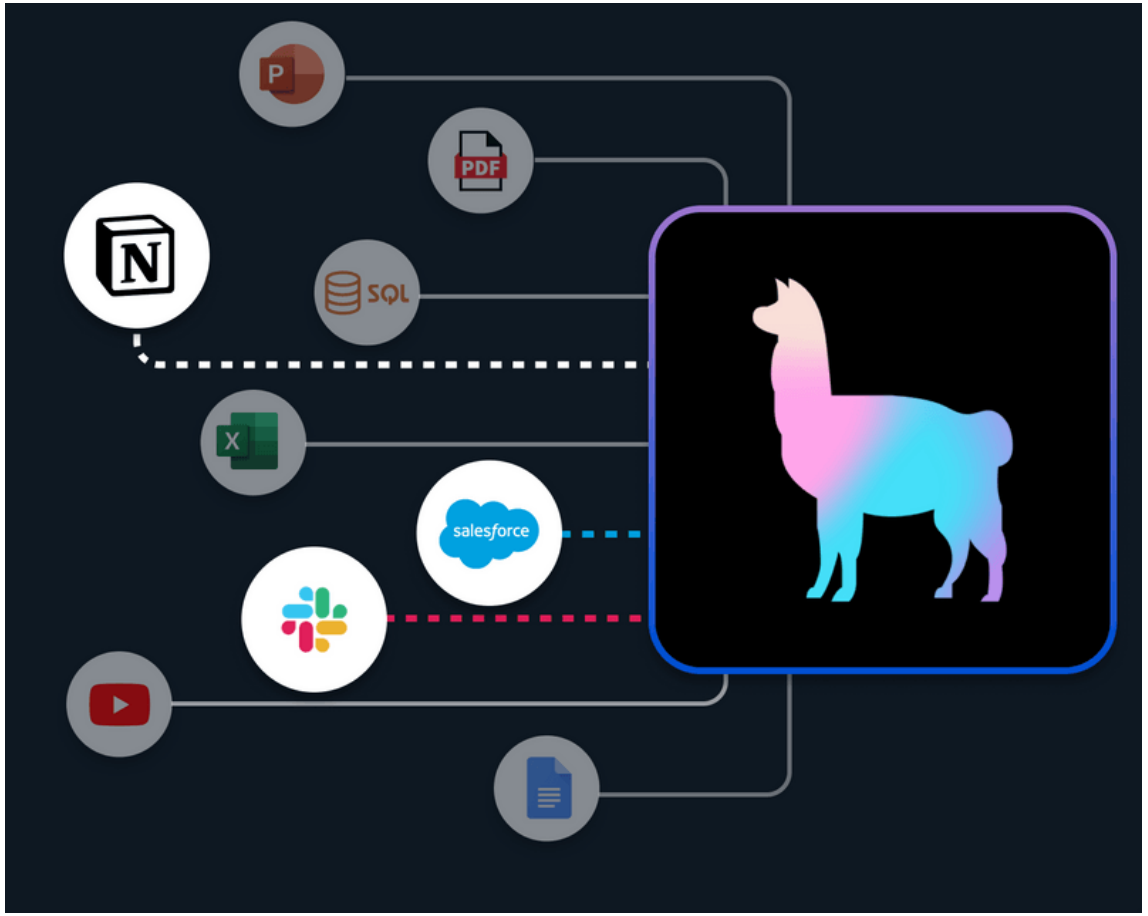
1. INTRODUCCIÓN A LOS ORQUESTADORES DE LLMS

🦜🔗 LangChain

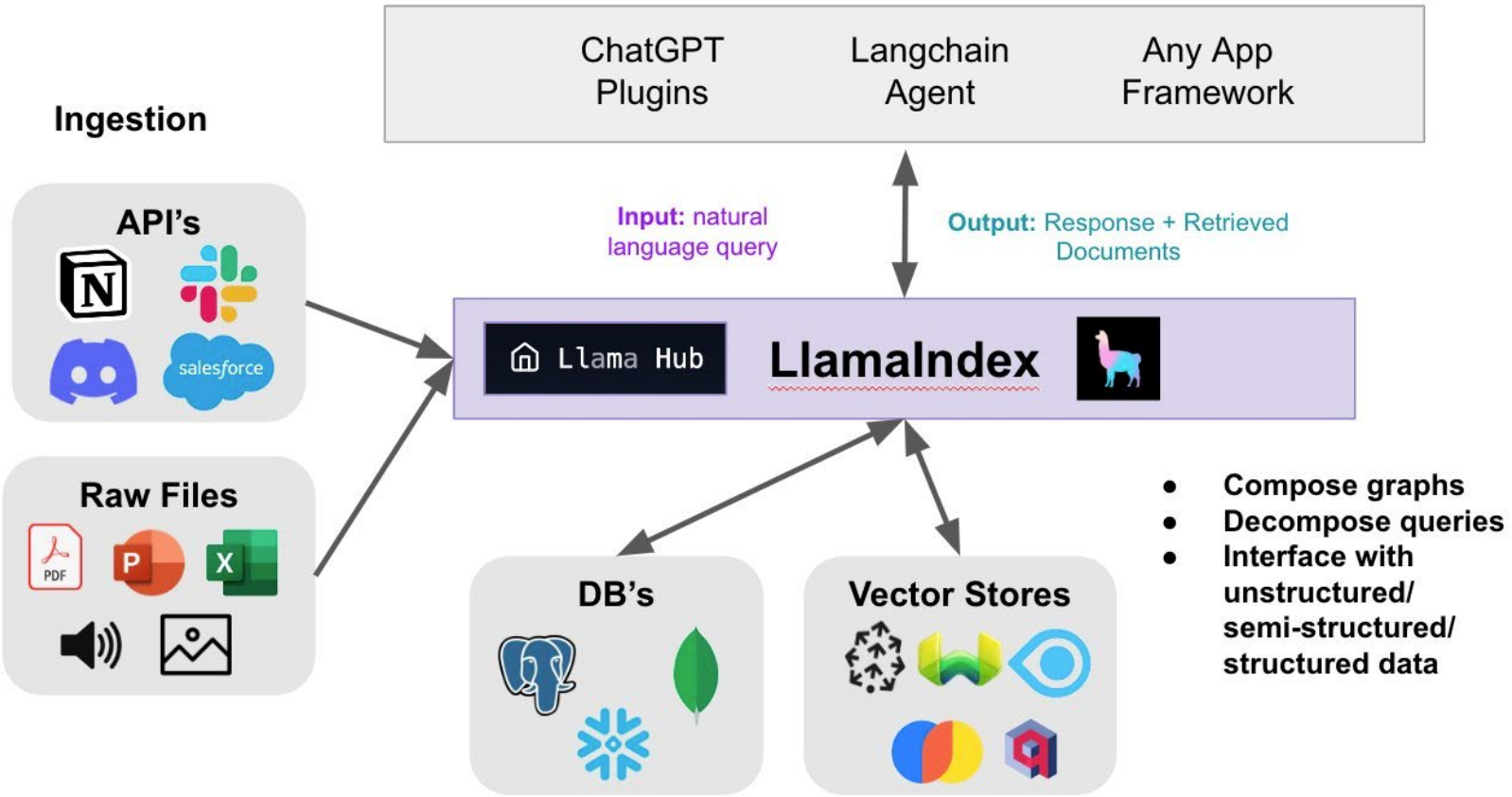
1. INTRODUCCIÓN A LOS ORQUESTADORES DE LLMS



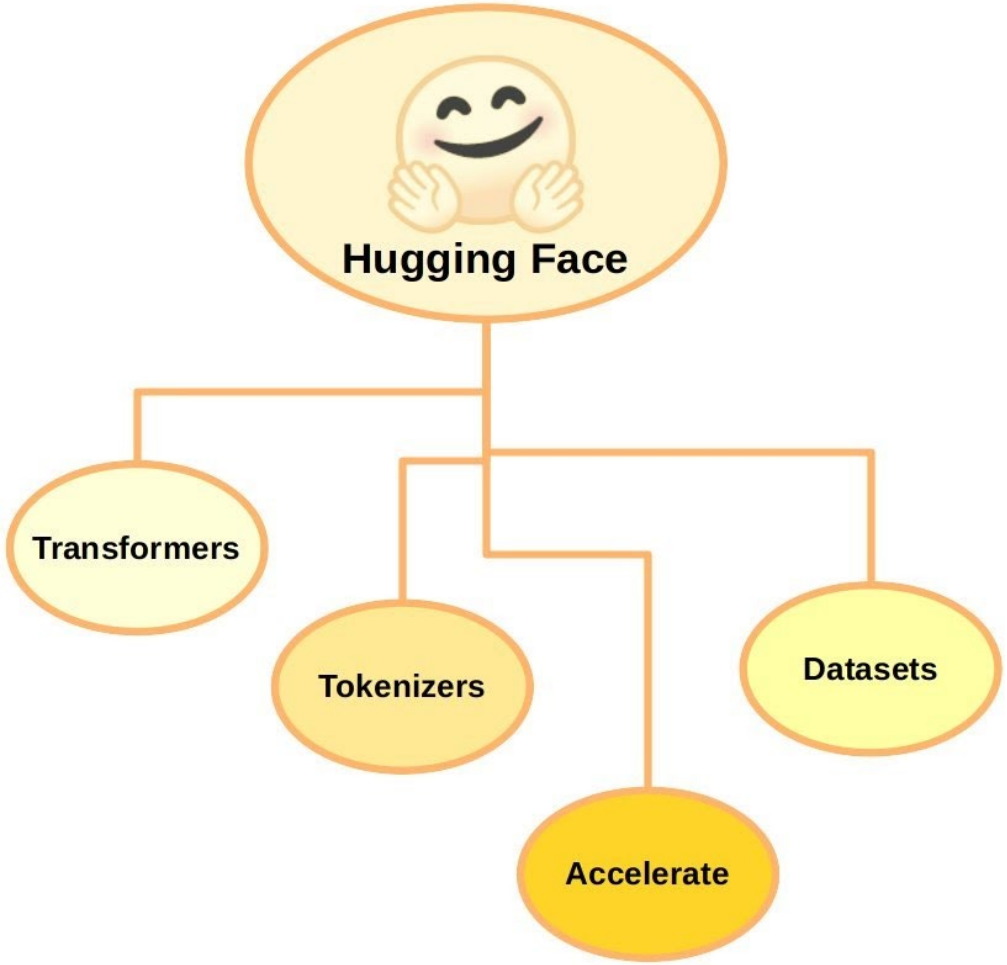
1. INTRODUCCIÓN A LOS ORQUESTADORES DE LLMS



1. INTRODUCCIÓN A LOS ORQUESTADORES DE LLMS

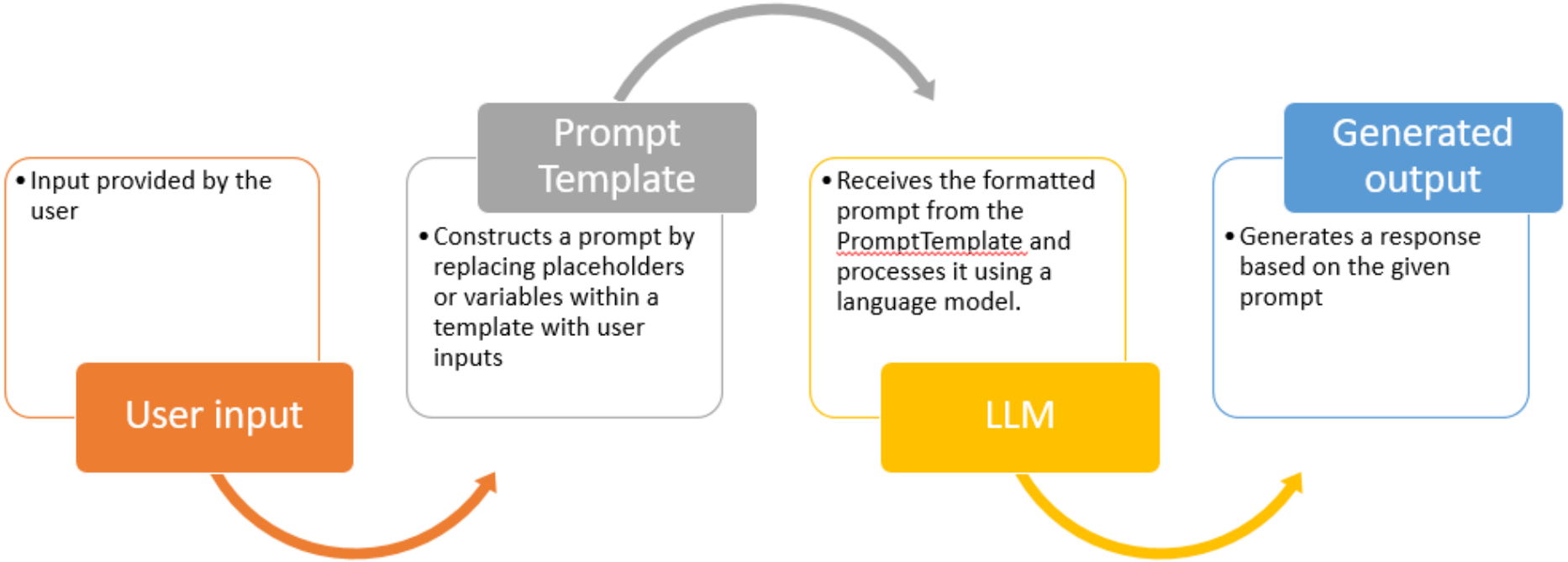


1. INTRODUCCIÓN A LOS ORQUESTADORES DE LLMS



2. CARACTERÍSTICAS Y FUNCIONALIDADES DE LANGCHAIN

3. CADENAS DE LANGCHAIN



LABORATORIO Nº 17: ORQUESTACIÓN CON LANGCHAIN

En este laboratorio, aplicarás tus conocimientos en FastAPI y Docker para desarrollar una aplicación que utilice el API de GPT y Redis, como sistema de caché para LLMs.

Tarea 1: Levantar una aplicación en FastAPI que realice una tarea de análisis sencilla mediante el API de GPT.

Tarea 2: Integrar una instancia de Redis mediante Docker Compose que funcione como un caché de solicitudes para el sistema de LLM.

Módulo 21: BD Vectoriales en sistemas basados en LLMs

AGENDA

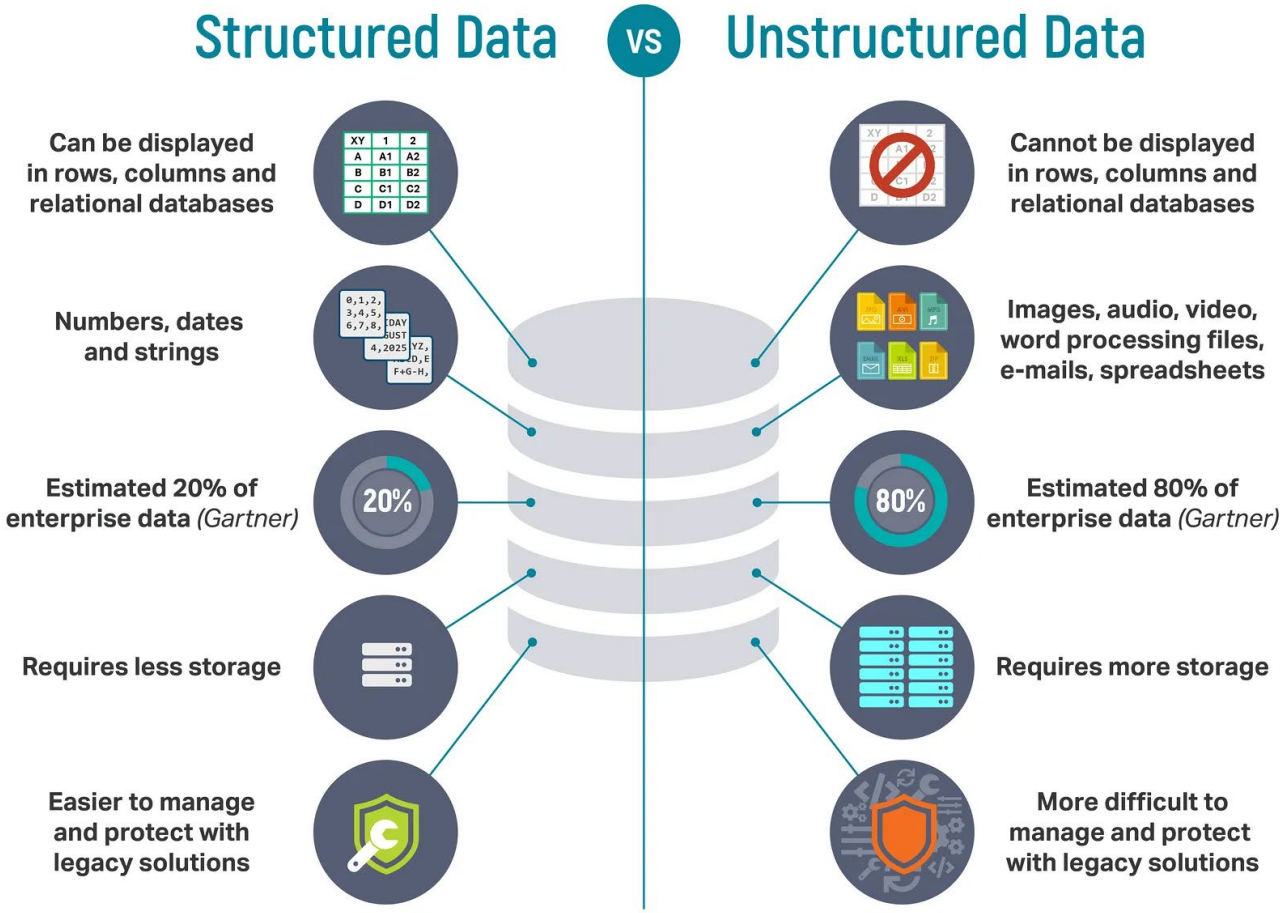
Introducción a las bases de datos vectoriales

1. Ventajas de las bases de datos vectoriales
2. Ejemplos de bases de datos vectoriales para LLMs
3. Casos de uso y aplicaciones

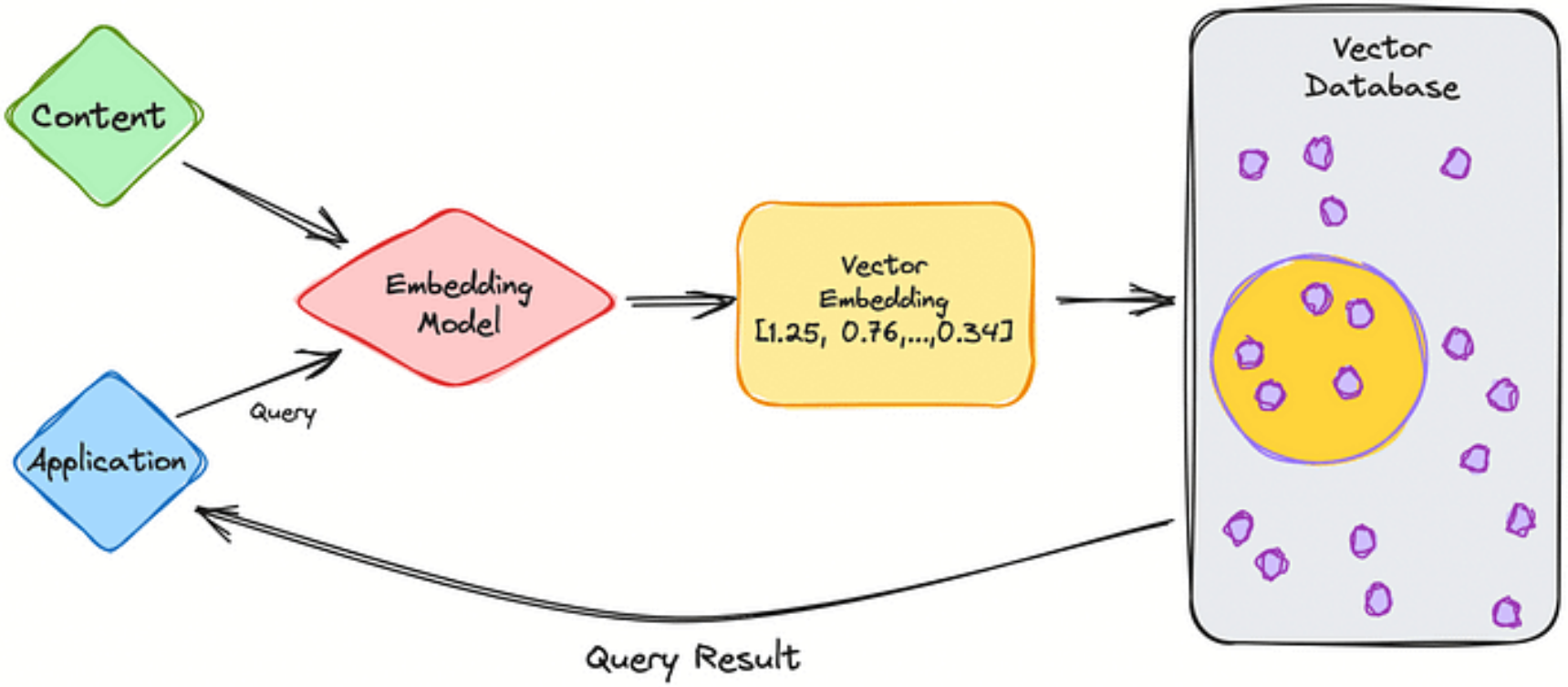
Laboratorio:

Sistema RAG con Langchain y Pinecone **(Evaluación)**

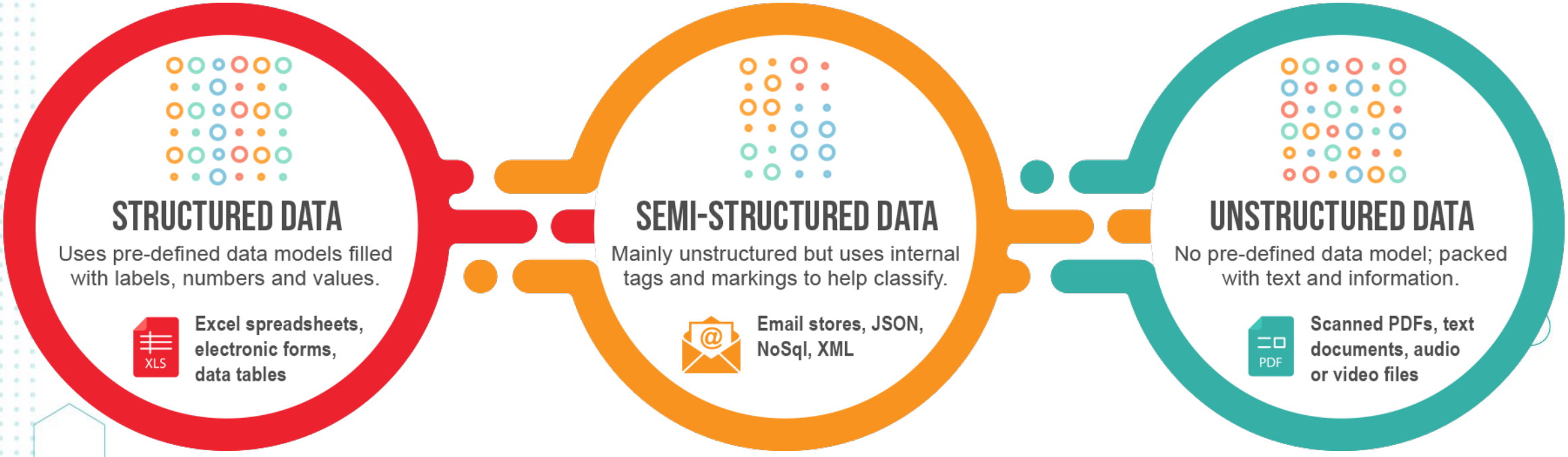
1. INTRODUCCIÓN A LAS BASES DE DATOS VECTORIALES



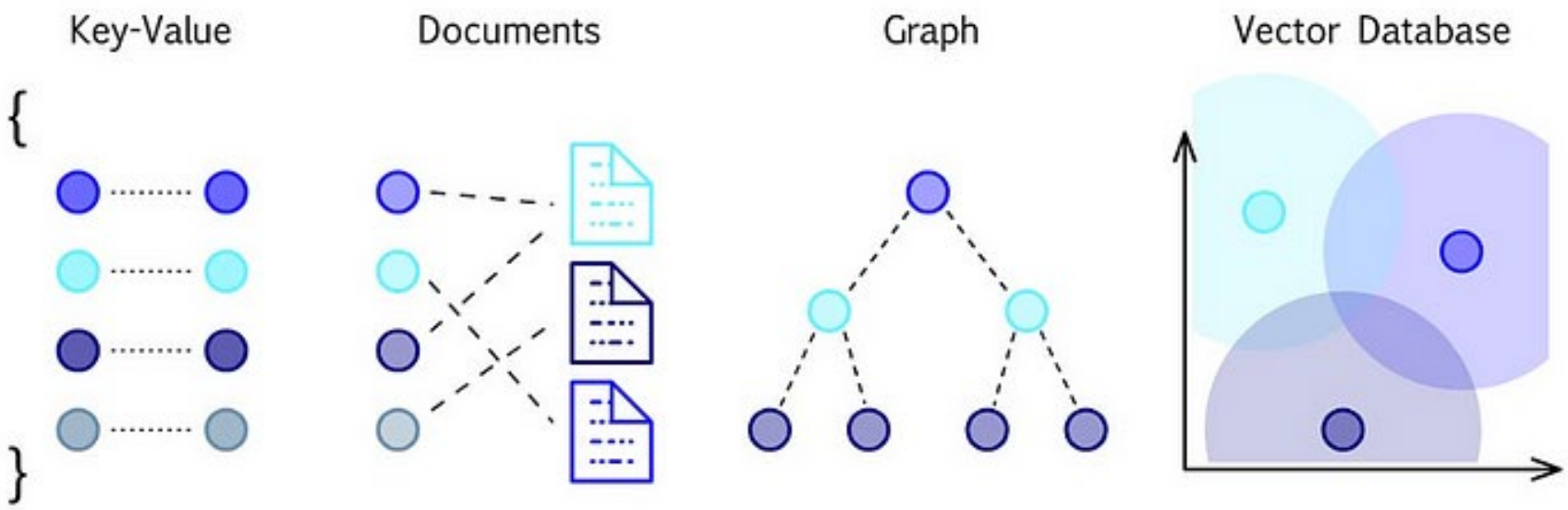
1. INTRODUCCIÓN A LAS BASES DE DATOS VECTORIALES



2. VENTAJAS DE LAS BASES DE DATOS VECTORIALES



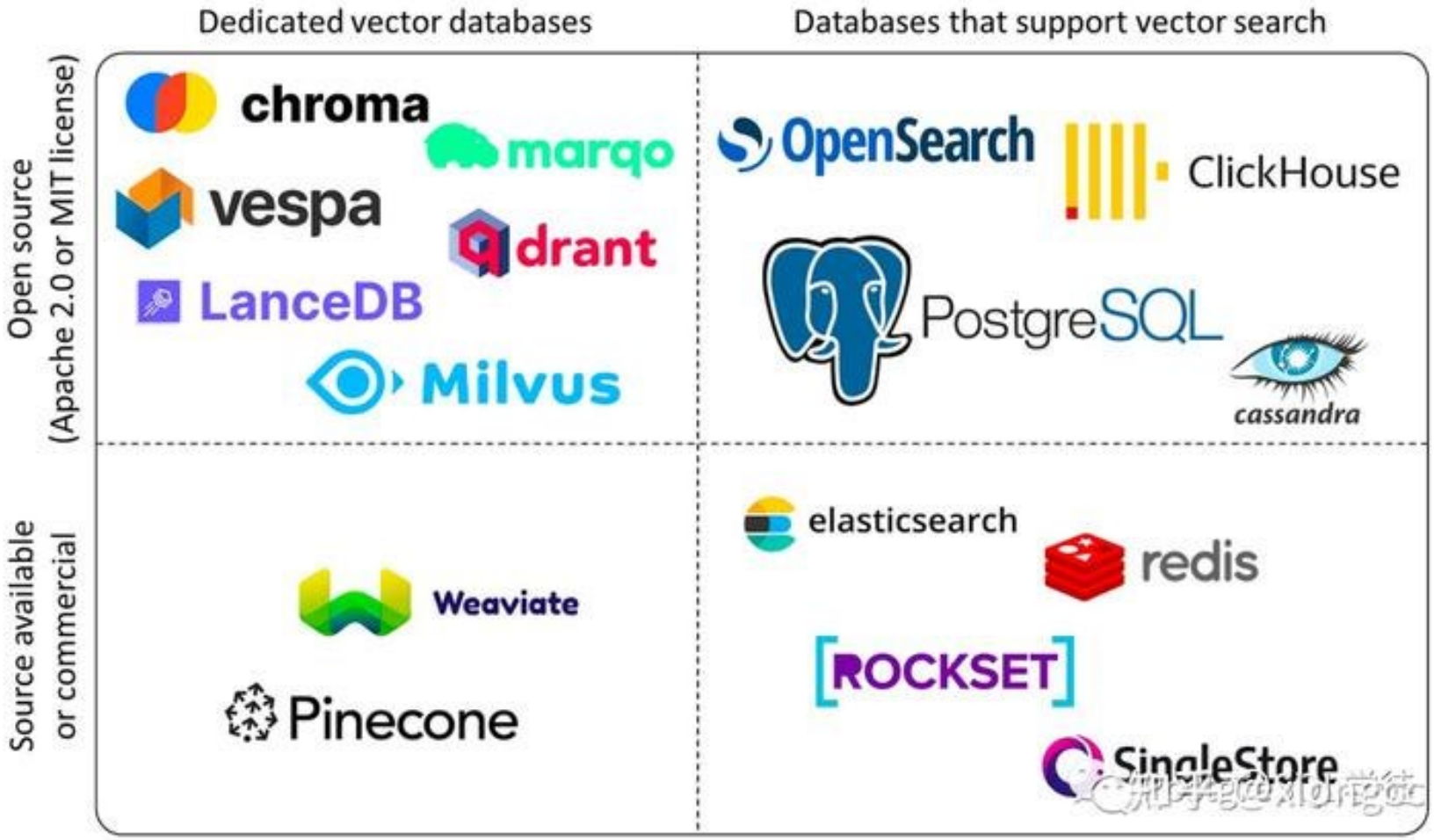
2. VENTAJAS DE LAS BASES DE DATOS VECTORIALES



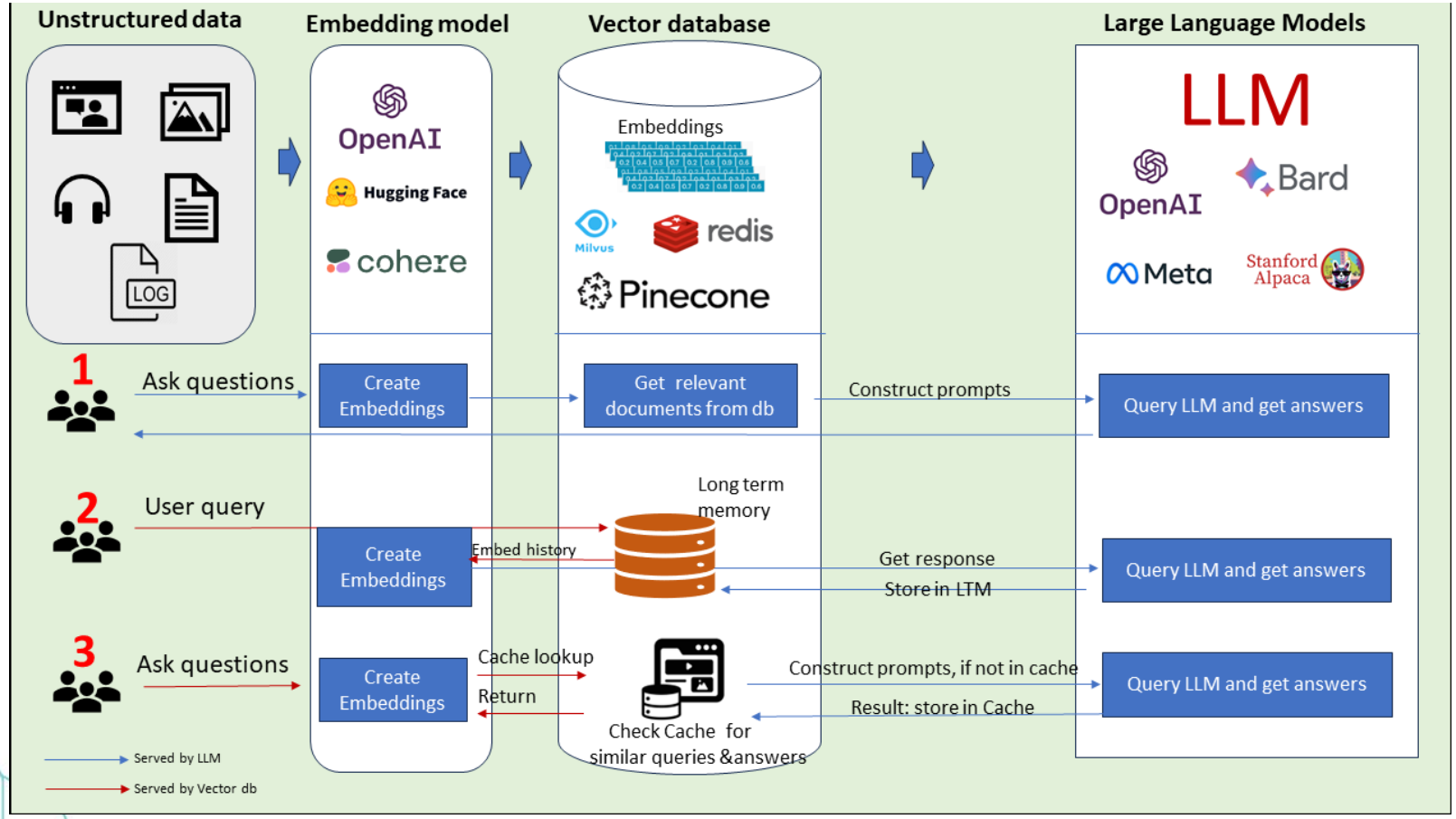
2. VENTAJAS DE LAS BASES DE DATOS VECTORIALES

- Manejo eficiente de datos de alta dimensión.
- Capacidades de búsqueda mejoradas.
- Integración mejorada del aprendizaje automático y la IA.
- Facilita análisis e información avanzados.

3. EJEMPLOS DE BASES DE DATOS VECTORIALES



4. CASOS DE USO Y APLICACIONES



LABORATORIO Nº 18: SISTEMA RAG CON LANGCHAIN Y PYTHON

En este laboratorio, podrás implementar una API en Python utilizando FastAPI y realizar una integración con una base de datos vectorial usando Pinecone.

Tarea 1: Implementar una API de búsqueda vectorial, incluyendo la carga de datos en la base.

Tarea 2: Integrar la API con el servicio Pinecone para gestionar la base de datos vectorial.

Tarea Opcional: Desarrollar un sistema RAG que integre la búsqueda vectorial con una consulta a OpenAI.

Módulo 22: Tools & Agentes

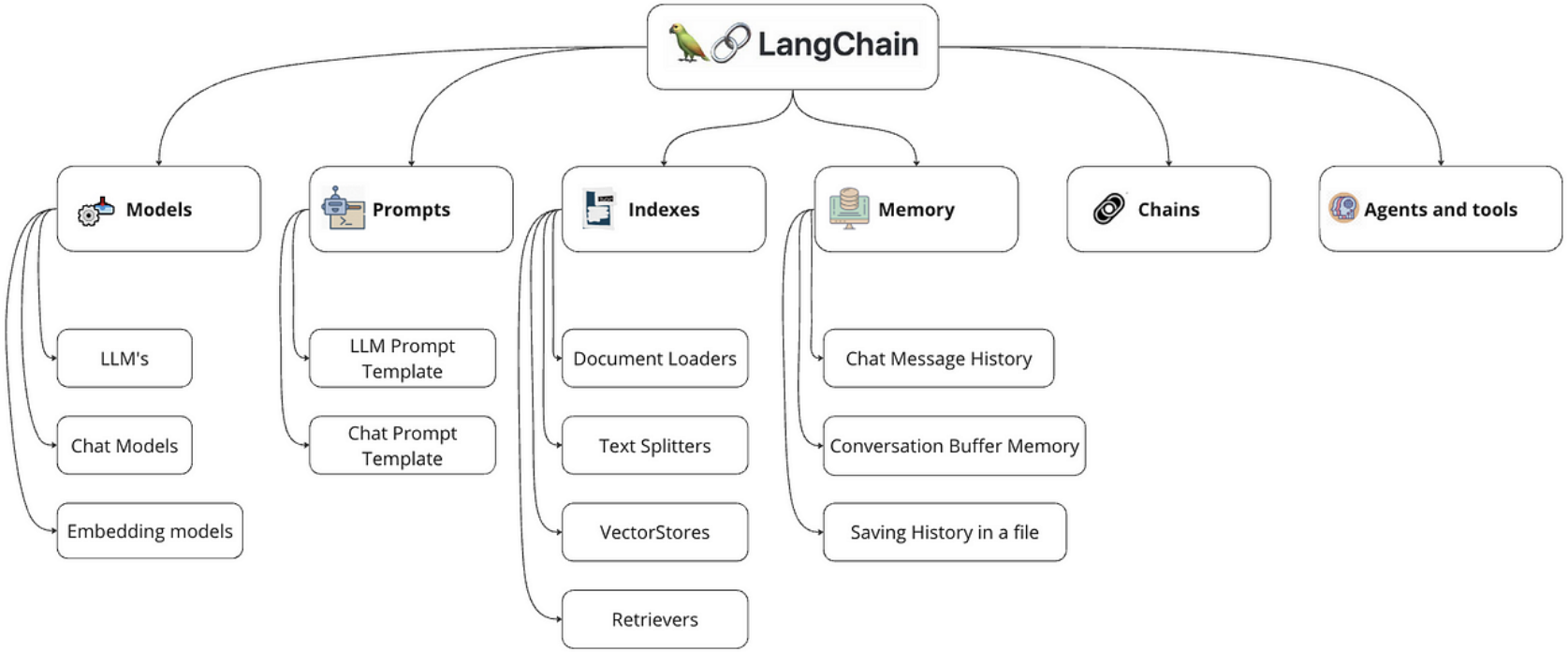
AGENDA

1. Langchain Agents
2. Langchain Tools

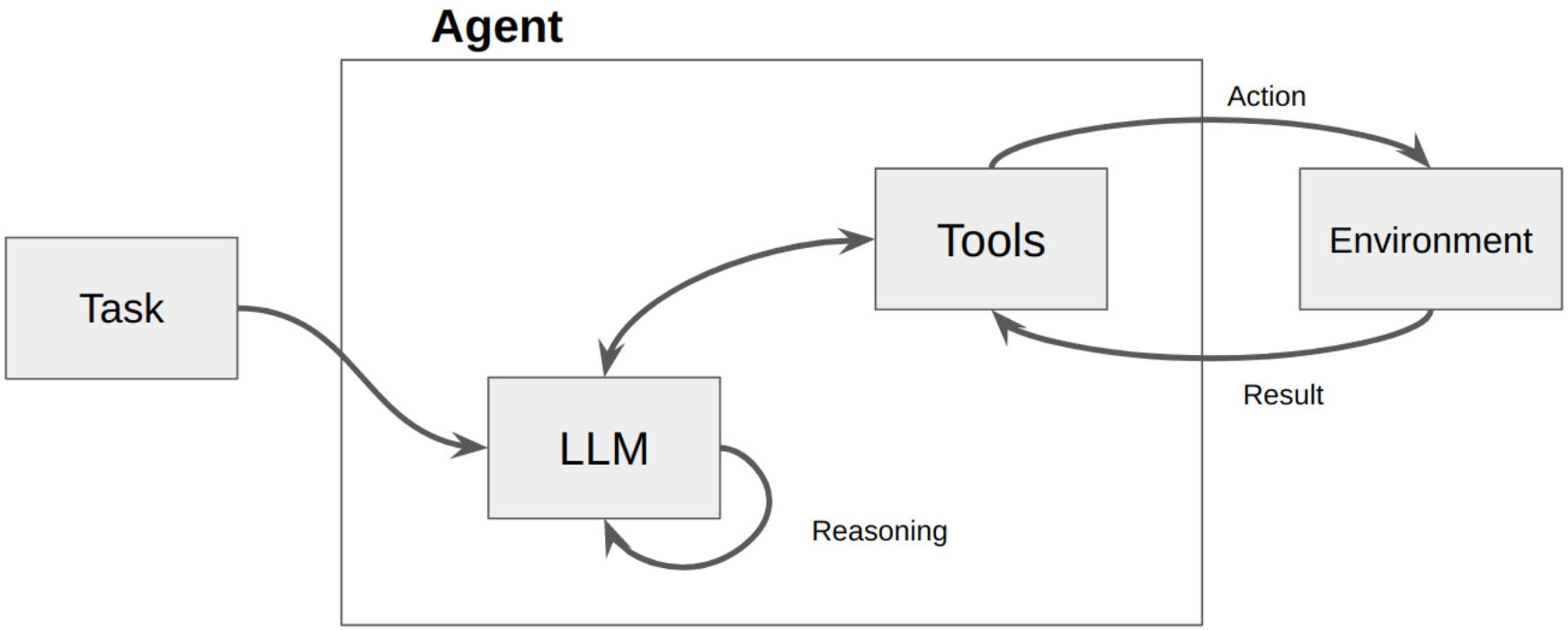
Laboratorio:

Sistema RAG con Langchain y Pinecone (Evaluación)

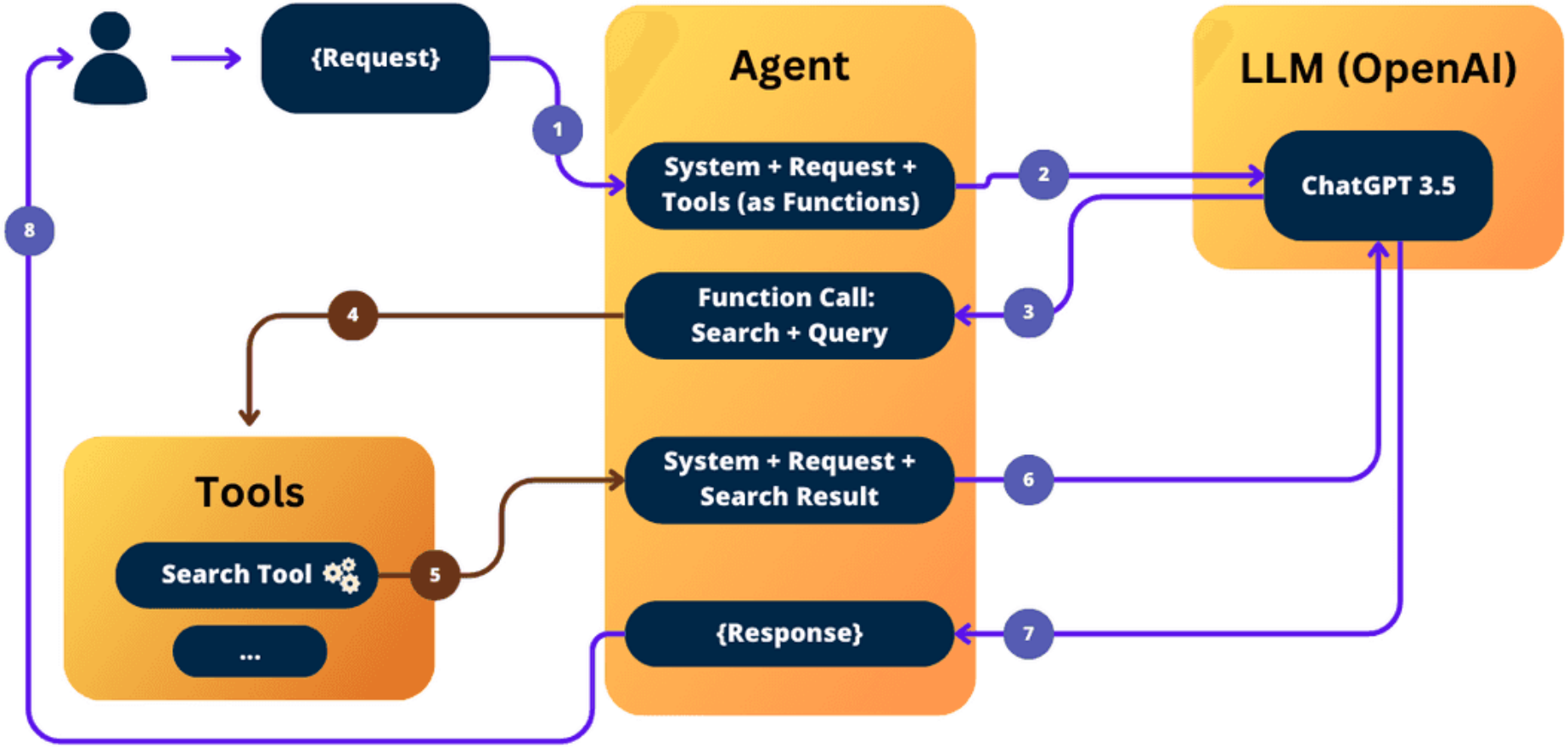
1. LANGCHAIN AGENTS



1. LANGCHAIN AGENTS



1. LANGCHAIN AGENTS



2. LANGCHAIN TOOLS

Shell (bash)

Giving agents access to the shell is powerful (though risky outsid...

Bearly Code Interpreter

Bearly Code Interpreter allows for remote execution of code. This

Bing Search

Microsoft Bing, commonly referred to as

Brave Search

This notebook goes over how to use the Brave Search tool. Go to ...

ChatGPT Plugins

This example shows how to use ChatGPT Plugins within LangChain

Connery Action Tool

Using this tool, you can integrate individual Connery Action into y...

Dall-E Image Generator

OpenAI Dall-E are text-to-image models

DataForSEO

DataForSeo provides comprehensive SEO and

LABORATORIO Nº 19: IMPLEMENTACIÓN DE AGENTS USANDO LANGCHAIN

En este laboratorio, podrás implementar y personalizar un agente de Langchain en Python, integrar una herramienta de búsqueda en Wikipedia y ejecutar consultas en bases de datos.

Tarea 1: Implementar y personalizar un agente de Langchain en un notebook de Python.

Tarea 2: Integrar una herramienta de búsqueda en Wikipedia al agente.

Tarea Opcional: Crear una herramienta que combine búsquedas en Wikipedia y consultas en bases de datos, comunicando los resultados al LLM.

Cierre de Unidad de Aprendizaje:
Integración de tecnologías del módulo en el proyecto del curso

TAREA Nº 6: DEFINICIÓN DE REQUERIMIENTOS. CONTENERIZACIÓN DE PROYECTO BASE

Al finalizar la tarea, lograrás:

- Definir requisitos del proyecto.
- Contenedorizar el proyecto base.
- Documentar requisitos funcionales y no funcionales.
- Configurar el entorno con Docker.
- Crear y gestionar contenedores.
- Preparar ambiente de desarrollo para implementación del proyecto.
- Mejorar portabilidad y gestión de recursos mediante contenedores.

LECTURAS ADICIONALES

Se sugiere revisar los siguientes enlaces para profundizar los conceptos tratados en el presente capítulo:

- Caching (2024). LangChain
  https://python.langchain.com/v0.1/docs/modules/model_io/llms/llm_caching/

- Optimizing LLMs for Speed and Memory (s.a) (s.f)
  https://huggingface.co/docs/transformers/llm_tutorial_optimization

- Schwaber, R. (2023). Qué son las bases de datos vectoriales y cómo pueden beneficiar a un sistema basado en IA (What is a Vector Database & How Does it Work? Use Cases + Examples | Pinecone)
  https://www.pinecone.io/learn/vector-database/

Guía de inicio con la librería LangChain

- Quickstart (2024). LangChain
  https://python.langchain.com/v0.1/docs/get_started/quickstart/

RESUMEN

En esta unidad, aprendiste:

- La contenedorización con Docker y docker-compose, diferenciando entre UI y API REST, utilizando herramientas como Streamlit y FastAPI.
- Habilidades para desplegar aplicaciones con Docker-compose y considerar mejores prácticas para el uso efectivo en entornos de desarrollo y producción.

CIBERTEC